



DevBurn: Proactive Machine Learning for Developer Burnout Prediction

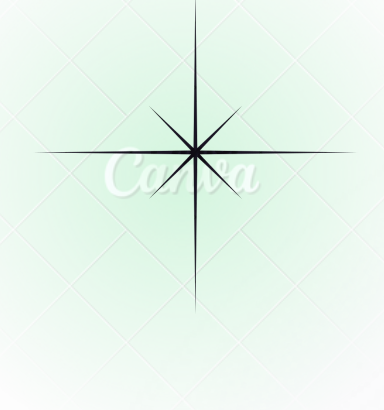
◆ **Course name: Software Development Management**

◆ **Group Number: A**

Md. Mehedi Hasan	04
Abdul Al Mahin	07
Marjanah Afrose	38

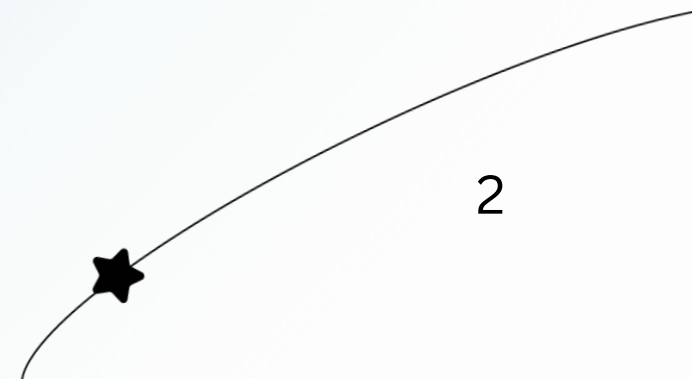
◆ **Course Teacher**

Md. Alamgir Hossain
Lecturer, Department of CSE.
Dhaka International University.



Content Outline

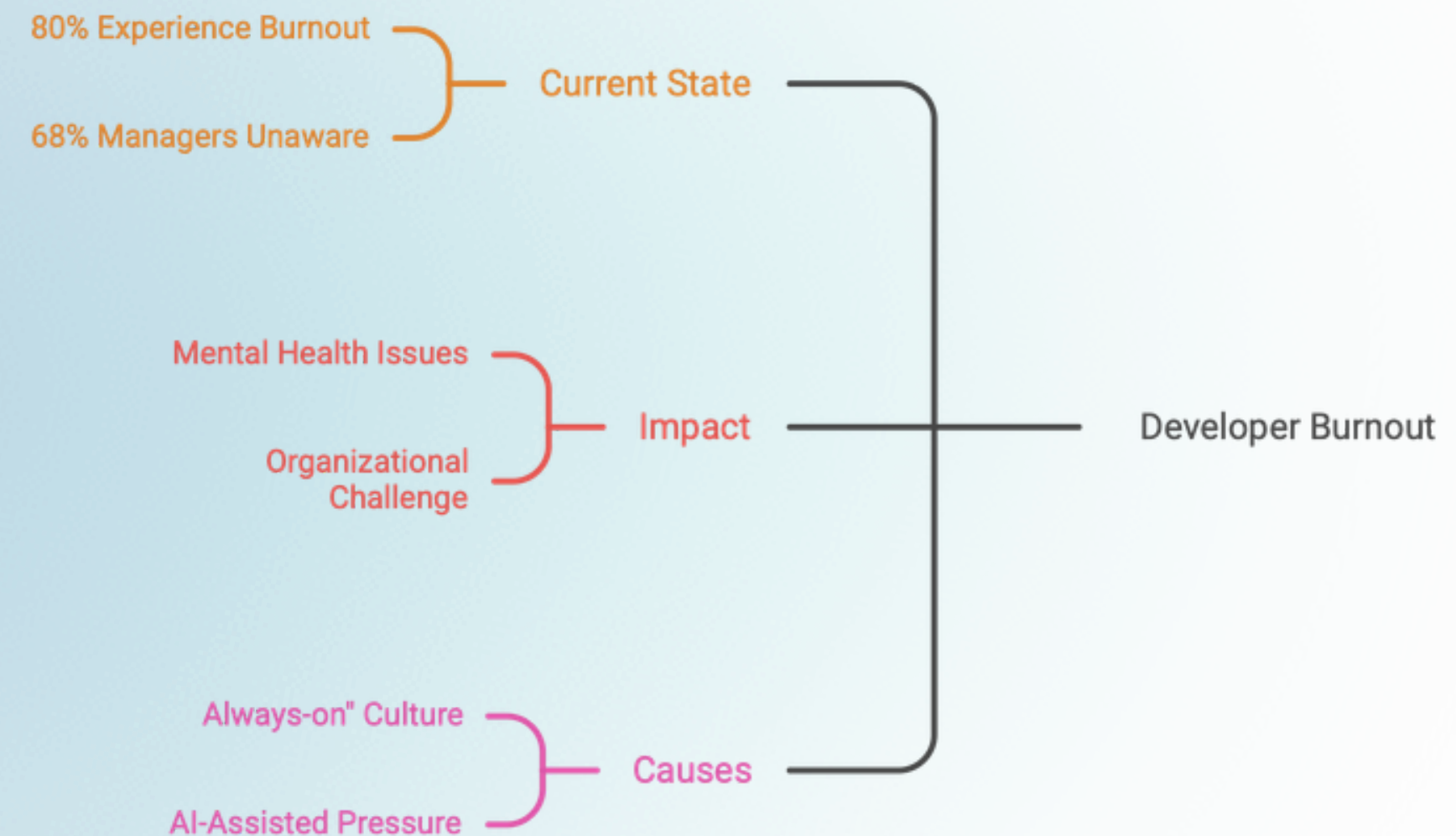
- | | | | | | |
|----|----------------|----|---------------------|----|---------------------|
| 01 | Introduction | 02 | Limitations | 03 | Research Gap |
| 04 | Key Indicators | 05 | System Architecture | 06 | Research Objectives |
| 07 | Contributions | 08 | Strategic Impact | 09 | References |



Introduction

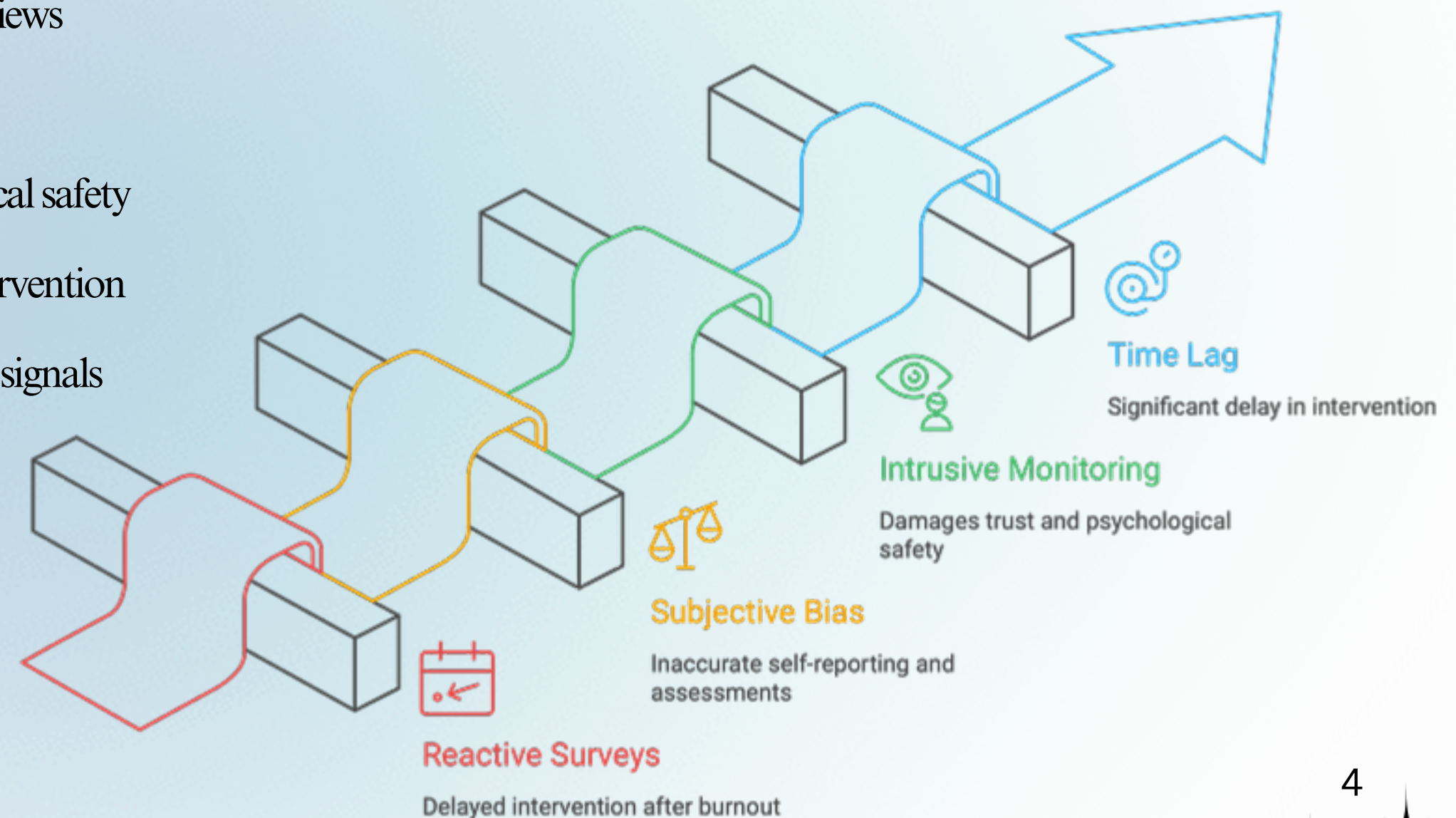
The Growing Crisis of Developer Burnout

- The current state of developer well-being in 2026
- Research indicates over 80% of software engineers experience significant burnout
- Invisible nature of burnout where 68% of managers remain unaware until resignation
- Impact of "Always-on" culture and AI-assisted productivity pressure on mental health



Limitations of Current Detection Methods

- Reactive nature of traditional surveys and annual performance reviews
- Subjective bias in self-reporting and manager assessments
- Intrusive monitoring techniques that damage trust and psychological safety
- Significant time lag between burnout onset and organizational intervention
- The critical need for objective, passive, and real-time early warning signals



Research Gap and DevBurn Innovation

- Absence of integrated systems using version control behavior for prediction [1]
- Innovation in utilizing passive GitHub activity signals without manual input [2]
- Continuous risk modeling versus static, one-time evaluations [3]
- Reframing burnout as a measurable engineering signal rather than a mystery [4]
- Foundation for the DevBurn system: Bridging behavioral science and data engineering [5]



Bridging Disciplines

Behavioral science and data engineering

Reframing Burnout

Burnout as a measurable signal

Continuous Risk Modeling

Static evaluations are insufficient

Passive Signal Innovation

Utilizing passive GitHub activity

Version Control Absence

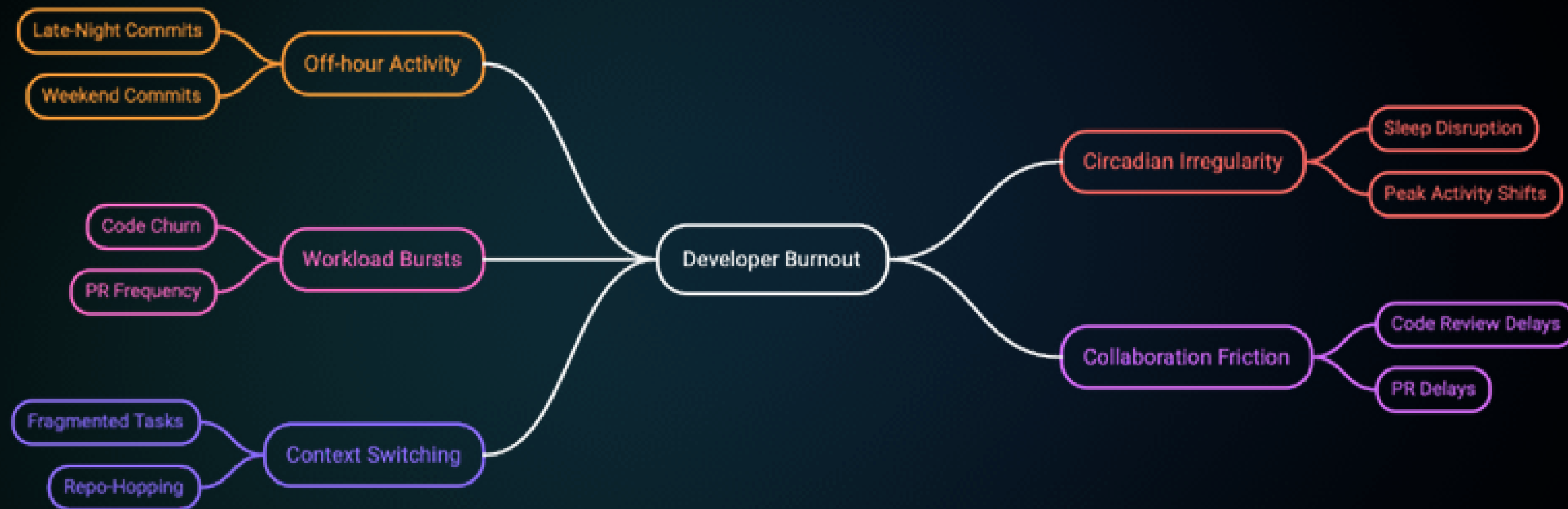
No version control behavior used

Key Behavioral Indicators on GitHub

- Off-hour activity: Sustained late-night or weekend commit patterns
- Circadian irregularity: Shifts in peak activity hours indicating sleep disruption
- Workload bursts: Sudden spikes in code churn and pull request frequency
- Collaboration friction: Increased lead time for code reviews and PR delays
- Context switching: High frequency of fragmented tasks and repo-hopping



Key Behavioral Indicators on GitHub



Proposed DevBurn System Architecture

- Data Acquisition: Non-intrusive extraction via GitHub APIs (Commits, PRs, Timestamps)
- Feature Engineering: Transforming raw activity into temporal and behavioral signals
- Machine Learning Engine: Utilizing Random Forest and XGBoost for risk scoring
- Actionable Intelligence: Translating complex data into a simplified team health dashboard
- Scalability and privacy-first design for enterprise deployment



DevBurn System Architecture



Data Acquisition

Non-intrusive
extraction via GitHub
APIs

Feature Engineering

Transforming raw
activity into temporal
and behavioral
signals

Machine Learning Engine

Utilizing Random
Forest and XGBoost
for risk scoring

Actionable Intelligence

Translating complex
data into a simplified
team health
dashboard

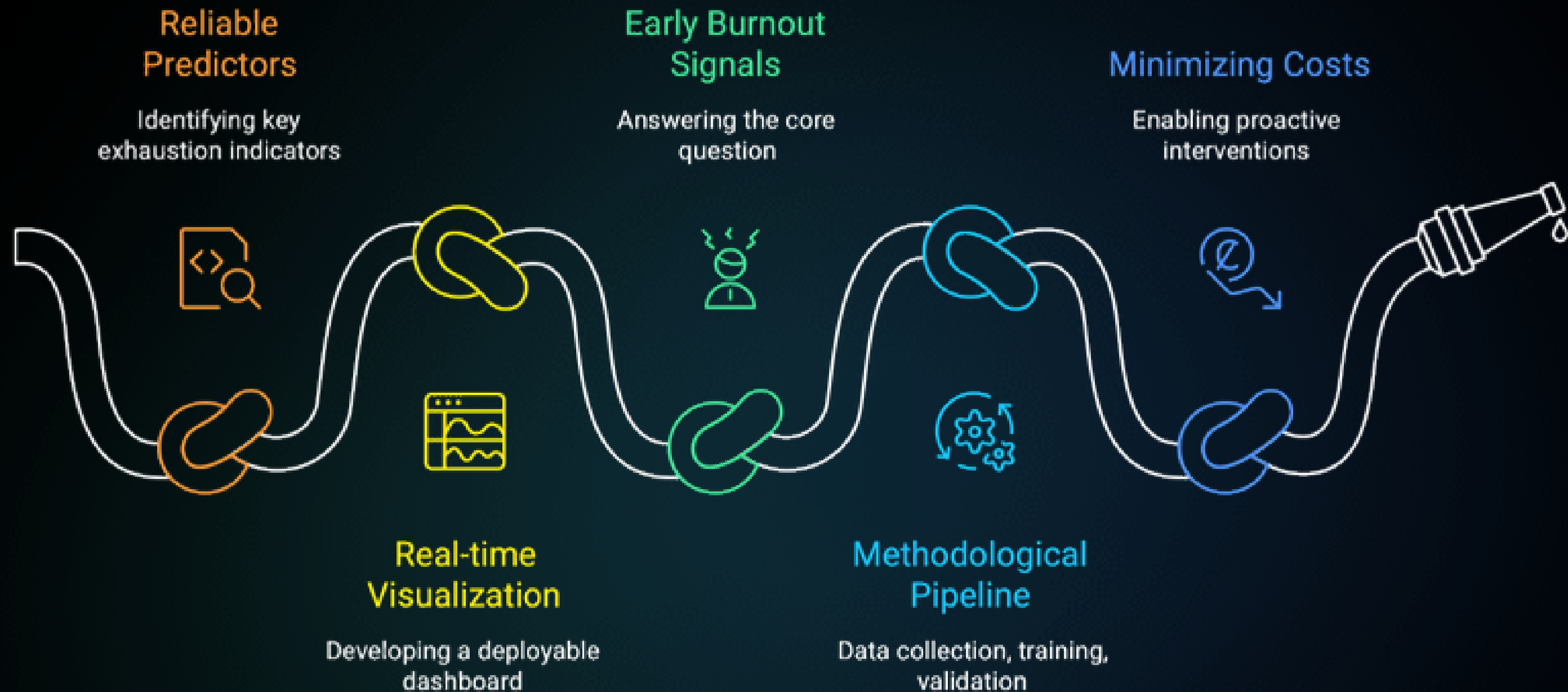
Scalability and Privacy

Designing for
enterprise
deployment with
privacy in mind

Research Objectives and Methodology

- Identifying the most reliable repository-level predictors of exhaustion
- Developing a deployable web-based dashboard for real-time visualization
- Answering the core question: Which patterns serve as early burnout signals?
- Methodological pipeline: Data collection, model training, and system validation
- Goal: Minimizing organizational costs by enabling proactive interventions

Addressing Software Developer Exhaustion



Unique Contributions to Software Engineering

- Unique Contributions to Software Engineering
- Introduction of behavioral feature innovation (e.g., activity bursts)
- Transitioning burnout detection from retrospective to proactive frameworks
- Elimination of reliance on invasive or subjective monitoring techniques
- Providing a bridge between academic research and industry practice
- Empowering engineering leaders with data-driven talent retention tools

DevBurn System Contributions



Strategic Impact and Future Outlook

- Reducing attrition rates through early stage risk identification
- Enhancing code quality by preventing cognitive overload in teams
- Optimizing work-life boundaries through evidence-based policy changes
- Future integration with GenAI adoption metrics and workplace toolchains
- Final takeaway: Burnout is a predictable phenomenon that requires systemic repair

From Burnout to Sustainable Development



Developer Burnout

High attrition, low quality, poor work-life balance

Early Risk Identification

Proactive identification of burnout risks

Cognitive Overload Prevention

Enhancing code quality and team efficiency

Work-Life Boundary Optimization

Evidence-based policy changes

GenAI Integration

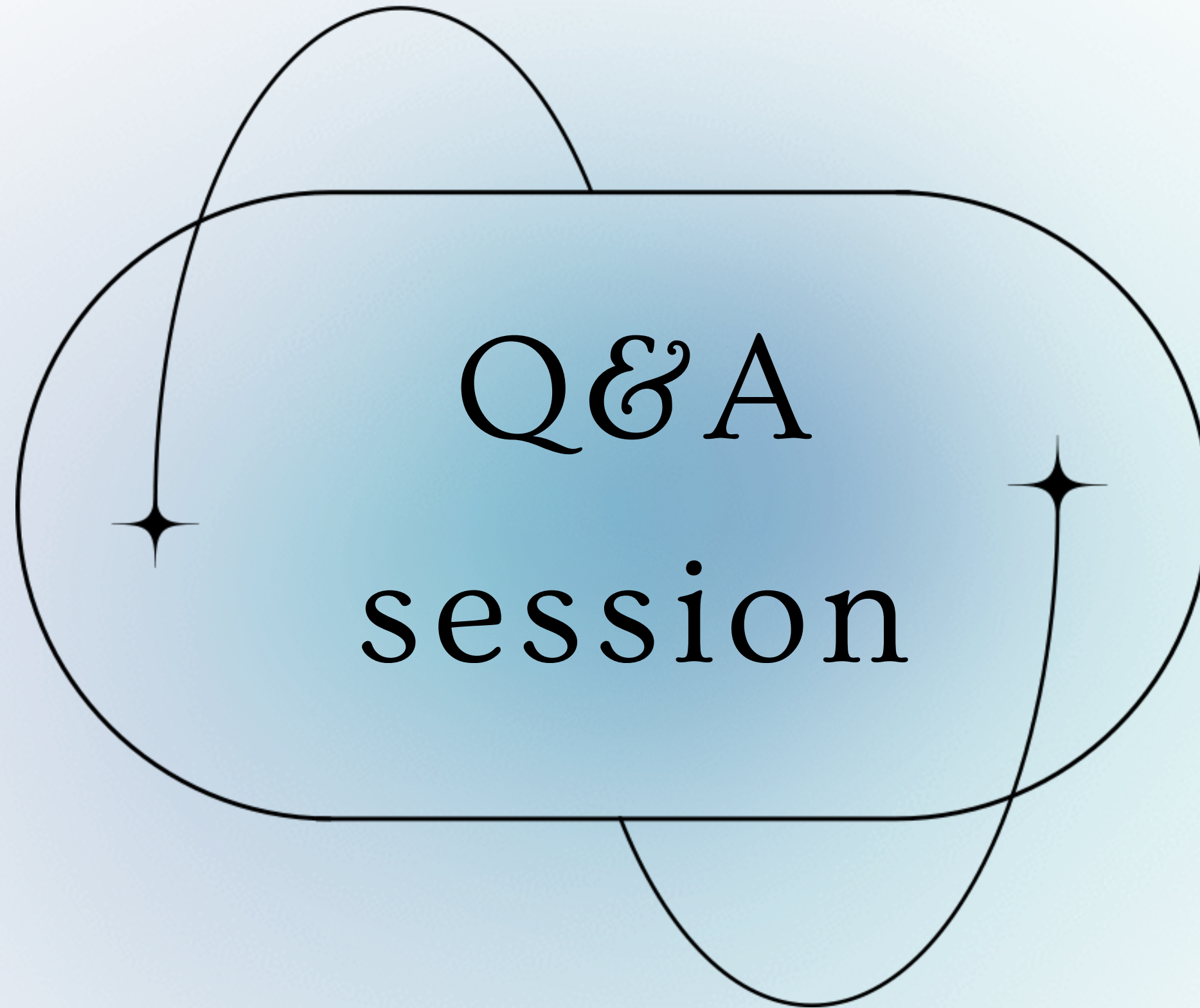
Future integration with AI adoption metrics

Sustainable Development

Reduced attrition, high quality, balanced life

Selected References

- Maslach & Leiter (2016): Understanding the burnout experience
- Alam & Ul Alam (2026): Explainable multitask burnout prediction (EMBRACE)
- Feng et al. (2025): Modeling developer burnout with GenAI adoption
- Tulili et al. (2026): ML models for early detection in software engineering
- Industry Reports (2025-2026): DORA and GitHub Innovation Graph insights





Thank you